

# torch.fft.hfft2#

<https://pytorch.org/docs/stable/generated/torch.fft.hfft2.html>

## Description:

Computes the 2-dimensional discrete Fourier transform of a Hermitian symmetric input signal. Equivalent to `hfft2()` but only transforms the last two dimensions by default. input is interpreted as a one-sided Hermitian signal in the time domain. By the Hermitian property, the Fourier transform will be real-valued. Supports `torch.half` and `torch.chalf` on CUDA with GPU Architecture SM53 or greater. However it only supports powers of 2 signal length in every transformed dimensions. With default arguments, the size of last dimension should be  $(2^n + 1)$  as argument `s` defaults to even output size =  $2 * (\text{last\_dim\_size} - 1)$  input (Tensor) – the input tensor

## Function Signature

---

```
torch.fft.hfft2(input, s=None, dim=(-2, -1), norm=None, *,  
out=None) → Tensor#
```

Parameters

Keyword Arguments

## Parameters

---

Keyword

out (Tensor, optional) – the output tensor.

## Code Examples

---

```
>>> torch.fft.hfft2(t).size()
torch.Size([10, 10])
```

```
>>> roundtrip = torch.fft.hfft2(t, T.size())
>>> roundtrip.size()
torch.Size([10, 9])
>>> torch.allclose(roundtrip, T)
True
```

## Notes & Warnings

---

**NOTE** Note Supports torch.half and torch.chalf on CUDA with GPU Architecture SM53 or greater. However it only supports powers of 2 signal length in every transformed dimensions. With default arguments, the size of last dimension should be  $(2^n + 1)$  as argument s defaults to even output size  $= 2 * (\text{last\_dim\_size} - 1)$

## Detailed Documentation

---

### Documentation

Computes the 2-dimensional discrete Fourier transform of a Hermitian symmetric input signal. Equivalent to `hfft2()` but only transforms the last two dimensions by default.

input is interpreted as a one-sided Hermitian signal in the time domain. By the Hermitian property, the Fourier transform will be real-valued.

Supports `torch.half` and `torch.chalf` on CUDA with GPU Architecture SM53 or greater. However it only supports powers of 2 signal length in every transformed dimensions. With default arguments, the size of last dimension should be  $(2^n + 1)$  as argument `s` defaults to even output size =  $2 * (\text{last\_dim\_size} - 1)$

input (Tensor) – the input tensor

s (Tuple[int], optional) – Signal size in the transformed dimensions. If given, each dimension `dim[i]` will either be zero-padded or trimmed to the length `s[i]` before computing the Hermitian FFT. If a length -1 is specified, no padding is done in that dimension. Defaults to even output in the last dimension: `s[-1] = 2*(input.size(dim[-1]) - 1)`.

dim (Tuple[int], optional) – Dimensions to be transformed. The last dimension must be the half-Hermitian compressed dimension. Default: last two dimensions.

norm (str, optional) –

Normalization mode. For the forward transform (`hfft2()`), these correspond to:

"forward" - normalize by  $1/n$

"backward" - no normalization

"ortho" - normalize by  $1/\sqrt{n}$  (making the Hermitian FFT orthonormal)

Where  $n = \text{prod}(s)$  is the logical FFT size. Calling the backward transform (`ihfft2()`) with

the same normalization mode will apply an overall normalization of  $1/n$  between the two transforms. This is required to make `ihfft2()` the exact inverse.

Default is "backward" (no normalization).

out (Tensor, optional) – the output tensor.

Starting from a real frequency-space signal, we can generate a Hermitian-symmetric time-domain signal: >>> T = torch.rand(10, 9) >>> t = torch.fft.ihfft2(T)

Without specifying the output length to `hfft2()`, the output will not round-trip properly because the input is odd-length in the last dimension:

So, it is recommended to always pass the signal shape `s`.